

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	5
2. РАБОЧЕЕ ПРОЕКТИРОВАНИЕ	8
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18

ВВЕДЕНИЕ

Современная разработка программного обеспечения всё активнее использует harness-инструменты – специализированные ИИ-ассистенты, встраиваемые непосредственно в рабочее окружение разработчика. К числу таких инструментов относятся Opencode, Cursor, GitHub Copilot и другие. В отличие от традиционных веб-чатов с искусственным интеллектом (таких как ChatGPT, Claude, DeepSeek), которые работают изолированно в окне браузера и требуют ручного копирования каждого фрагмента кода, harness-инструменты имеют прямой доступ к файловой системе проекта, могут читать, создавать и модифицировать файлы, выполнять произвольные команды в терминале, работать с системой контроля версий Git и запускать локальные серверы для тестирования в реальном времени.

Принципиальное различие между веб-чатом и harness-инструментом заключается в уровне интеграции. Веб-чат – это изолированный консультант: разработчик задаёт вопрос, получает ответ в виде текста и должен самостоятельно применить его к проекту. Harness-инструмент, напротив, действует как полноценный ассистент: он видит всю структуру проекта, понимает взаимосвязи между файлами, может выполнять многошаговые операции и самостоятельно проверять результат. Например, если требуется изменить физику прыжка персонажа, веб-чат предложит фрагмент кода, который нужно вручную найти и заменить в нужном файле. Harness-инструмент же сам найдёт файл с классом Player, определит текущие значения параметров, изменит их, проверит синтаксис и запустит проект для тестирования.

Экономия времени при использовании harness-инструментов достигается за счёт автоматизации рутинных операций: создание файлов, форматирование кода, исправление синтаксических ошибок, генерация документации, выполнение git-команд, запуск сервера. В сложных многофайловых проектах, особенно на начальных этапах разработки, эта

экономия может составлять до 80–90% времени по сравнению с ручной реализацией или использованием веб-чата.

Важной особенностью harness-инструментов является возможность работы в двух режимах: Plan mode (режим планирования), где инструмент помогает составить архитектурный план и структуру проекта, и Build mode (режим сборки), где он непосредственно реализует запланированные компоненты. Это позволяет сначала тщательно продумать архитектуру, затем поэтапно внедрять её без потери контекста и при необходимости возвращаться к плану для внесения изменений.

Данная работа посвящена практическому применению harness-инструмента Opencode для разработки веб-игры «Doodle Jump» – вертикального 2D-платформера, в котором персонаж автоматически прыгает по платформам, продвигаясь вверх, а игрок управляет его горизонтальным перемещением. В рамках работы harness-инструмент использовался на всех этапах: от создания начального каркаса проекта до финальной полировки, отладки и публикации на GitHub Pages.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Целью данной работы является разработка веб-игры в жанре 2D-платформера с использованием готовых ассетов. Игра «Doodle Jump» представляет собой браузерную адаптацию классической аркадной игры: персонаж (Doodle) автоматически прыгает по платформам, поднимаясь всё выше; игрок управляет только горизонтальными перемещениями с помощью стрелок или клавиш A/D. Задача игрока – забраться как можно выше, набирая очки, избегая врагов и собирая бонусы.

Проект имеет следующую структуру:

- `index.html` (20 строк) – основная HTML-страница с элементом `canvas` для рендеринга, подключающая пять JavaScript-модулей через теги `script` в порядке `platform.js`, `enemy.js`, `player.js`, `game.js`, `script.js`.

- `style.css` (21 строка) – стили для полноэкранного отображения `canvas`, тёмный фон (`#1a1a2e`), отключение скrolла и центрирование по горизонтали.

- `script.js` (15 строк) – точка входа приложения: инициализация класса `Game`, динамическое изменение размера `canvas` под высоту окна (ширина фиксирована `400px`, высота равна `window.innerHeight`), запуск игрового цикла через `game.init()`.

- `game.js` (1322 строки) – ядро игры: управление состояниями (меню, игра, пауза, `Game Over`, настройки), генерация уровней с тремя конфигурациями сложности, коллизии (AABB), рендеринг с вертикальным масштабированием, система бонусов (джетпак, щит, монеты трёх видов), спавн врагов с проверкой на перекрытие с игроком, отрисовка интерфейса (HUD, меню, `Game Over`, настройки, пауза), звуковая подсистема (3 звуковых эффекта), система жизней (3 жизни), сохранение рекорда и настроек громкости в `localStorage`.

- `player.js` (257 строк) – класс `Player`, реализующий физику движения: гравитация `0.5`, сила прыжка `-12`, сила прыжка с пружины `-18`,

горизонтальная скорость 5, затухание 0.8. Хитбокс персонажа – 30×30 пикселей. Поддержка джетпака ($v_y = -3$, 120 кадров), щита (1 блокировка врага), админ-режима (F2, свободный полёт со скоростью 8). Два кадра анимации.

- platform.js (80 строк) – класс Platform с четырьмя типами: green (обычные статичные, 40% с декоративной травой), brown (разрушаемые, трескаются через 30 кадров после приземления), blue (пружинящие, усиленный прыжок –18, 30% с декоративным грибом), purple (движущиеся, скорость 1–1.5, отражение от краёв). Размер платформы – 70×16 пикселей.

- enemy.js (136 строк) – класс Enemy (горизонтальное перемещение с отражением от краёв, 2 кадра анимации, размер 28×28) и Coin (три типа: bronze +10 очков, silver +25, gold +50, анимация 4 кадра, плавающая высота с bobTimer, размер отображения 32×32).

Разработка такой игры является сложной задачей для обычного ИИ-веб-чата по нескольким причинам. Проект состоит из множества взаимосвязанных файлов: изменения в игровой логике (game.js) требуют синхронных правок в классах персонажей (player.js), врагов (enemy.js) и платформ (platform.js). Координация пяти модулей через копирование текста в веб-чат крайне неудобна и ведёт к потере контекста. Кроме того, локальные пути к ассетам должны быть корректно прописаны в коде – ошибка в пути приводит к тому, что игра не загружает спрайты. Разработка игры требует постоянного цикла «редактирование – тестирование – отладка»: нужно внести изменения в код, запустить локальный HTTP-сервер, открыть игру в браузере, проверить поведение, обнаружить ошибку и вернуться в код. Без прямого доступа к терминалу и файловой системе каждый такой цикл может занимать от 10 до 30 минут.

Использование harness-инструмента решает все перечисленные проблемы. Opencode имеет полный доступ к файловой системе, может выполнять команды в терминале (включая запуск HTTP-сервера и операции с Git), а главное – понимает контекст всего проекта одновременно, что позволяет

ему корректно координировать изменения во всех модулях. Более того, инструмент способен выполнять комплексные задачи, такие как «найти все файлы, использующие параметр скорости», за одну команду.

2. РАБОЧЕЕ ПРОЕКТИРОВАНИЕ

Разработка игры «Doodle Jump» проводилась с использованием harness-инструмента Opencode в несколько последовательных этапов.

Шаг 1 – создание папки проекта и инициализация структуры. Была создана корневая папка doodle, в которую впоследствии были добавлены все файлы игры, презентаций и ассетов. Opencode был запущен из этой директории, что обеспечило корректную работу со всеми относительными путями и возможность напрямую обращаться к файлам через инструменты Files и Shell.

Шаг 2 – загрузка графических и звуковых ассетов. Ассеты были скачаны с открытых источников (kenney.nl – рекомендуемый источник для 2D- и 3D-ассетов) и загружены в папку assets с организованной структурой. Внутри папки были созданы поддиректории: background (фон и облака), bonuses (монеты трёх видов с анимацией 4 кадра, иконки джетпака и щита, текстуры бонусов на персонаже), decor (декоративные элементы: трава двух видов, грибы красные и коричневые), enemies (два кадра анимации врага), platforms (четыре типа платформ: зелёные, коричневые, синие, фиолетовые), player (три состояния персонажа: jump1, jump2, fall), ui (иконки сердец full/empty и монет bronze/silver/gold для интерфейса), sounds (звуковые эффекты: jump, death, hit). Всего в папке assets размещено 38 PNG-изображений и 2 MP3-файла.

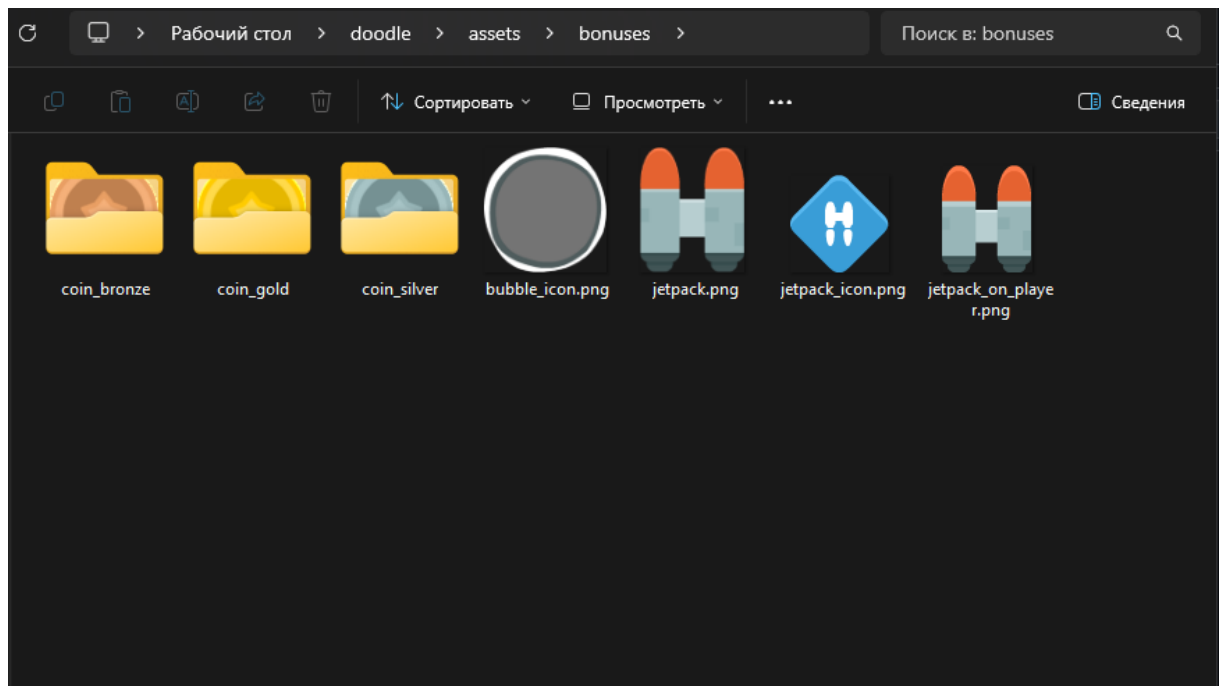


Рисунок 1 – Папка с ассетами

Шаг 3 – подготовка плана в Plan mode. Перед началом реализации в Opencode был активирован Plan mode – режим планирования и исследования, в котором инструмент может анализировать задачу, но не имеет доступа к выполнению действий (Files, Shell, Git и другие инструменты заблокированы). В этом режиме был составлен план MVP (Minimum Viable Product) игры, включающий разбивку на логические этапы: создание базовой HTML-страницы с canvas, реализация класса игрока с физикой движения и гравитацией, реализация платформ с процедурной генерацией, добавление врагов и системы коллизий, внедрение системы жизней и бонусов, разработка пользовательского интерфейса (меню, Game Over, настройки), публикация на GitHub Pages.

Шаг 4 – доработка плана. После обсуждения план был скорректирован с учётом требований к трём уровням сложности, системе монет трёх видов (бронзовые, серебряные, золотые с разной стоимостью), бонусам (джетпак на 120 кадров, щит на одну блокировку), вертикальному масштабированию canvas под высоту окна браузера и системе трёх жизней с режимом неуязвимости после получения урона.

Шаг 5 – переход в Build mode и реализация. После утверждения плана был активирован Build mode – рабочий режим с полным доступом ко всем инструментам. Harness-инструменту был дан доступ к разработке, и реализация велась поэтапно: на каждом этапе формулировалась узкая задача, инструмент выполнял её, после чего проводилась ручная проверка работоспособности в браузере. Проект изначально был задуман с модульной архитектурой, чтобы избежать монолитного файла на тысячи строк, в котором ИИ чаще ошибается при фиксах и доработках. Минимальная архитектура была реализована по схеме: index.html + style.css + script.js (подключение и запуск) + game.js (ООП-класс игры) + вспомогательные модули.

Шаг 6 – контроль реализации и итеративные улучшения. После каждого завершённого этапа проводилось тестирование игры в браузере с открытой консолью DevTools. При обнаружении ошибок или несоответствий формулировалась точная задача на исправление, после чего инструмент вносил точечные правки без переписывания всего проекта. В процессе разработки были последовательно реализованы: генерация платформ с возрастанием сложности, физика прыжка с гравитацией и пружинами, коллизии (AABB) с проверкой на приземление сверху, система жизней с неуязвимостью после получения урона, процедурный спавн врагов с проверкой на перекрытие с игроком, анимированные монеты с плавающей высотой, бонусы с таймерами и визуальной индикацией, экран Game Over с детализацией монет.

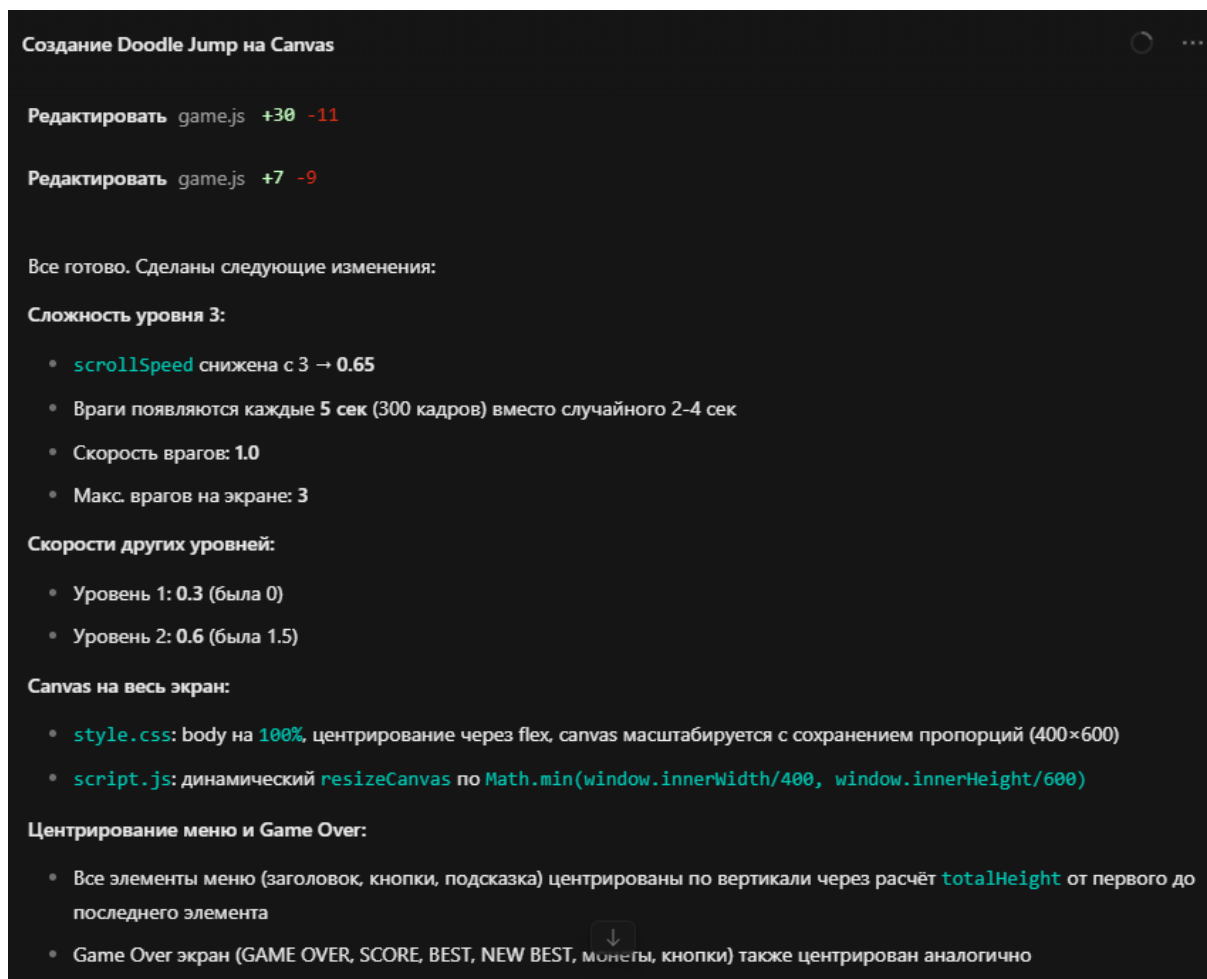


Рисунок 2 – Разработка игры в harness-инструменте

Шаг 7 – фиксация изменений в Git. После прохождения каждой контрольной точки выполнялся `git commit` с осмысленным сообщением, отражающим суть проделанной работы. После серии коммитов выполнялся `git push` в удалённый репозиторий на GitHub по адресу https://github.com/xndxrground/doodle_jump_replica.

Шаг 8 – публикация на GitHub Pages. Проект был выложен на GitHub Pages для обеспечения доступа к игре через браузер по постоянной ссылке https://xndxrground.github.io/doodle_jump_replica/. Для публикации использовалась стандартная инструкция: проверка статуса репозитория (Public), переход в Settings → Pages, выбор ветки main, ожидание сборки в течение примерно 5 минут.

Итоговая версия игры имеет следующие ключевые характеристики.

Игровой мир имеет виртуальный размер 400×600 пикселей. Canvas растягивается на всю высоту окна браузера: ширина canvas фиксирована (400px), высота равна window.innerHeight (переменная). Рендеринг игрового мира выполняется с вертикальным масштабированием: перед отрисовкой применяется `ctx.scale(1, this.height / this.gameHeight)` для заполнения всей доступной области экрана. UI-элементы (меню, счёт, Game Over, настройки, пауза) отрисовываются после снятия трансформации – в нативном разрешении canvas с фиксированными размерами шрифтов, что обеспечивает их чёткость независимо от масштабирования игрового мира.

В игре реализовано три уровня сложности, каждый со своими параметрами. Level 1 (лёгкий): скорость прокрутки 0.3, враги появляются раз в 600 кадров (10 секунд), максимум 2 врага, скорость врагов 0.5, движущиеся и пружинящие платформы отсутствуют, только зелёные платформы. Level 2 (средний): скорость прокрутки 0.6, враги раз в 360 кадров (6 секунд), максимум 3, скорость 0.8, появляются пружинящие (20%) и разрушаемые (20%) платформы. Level 3 (сложный): скорость прокрутки 0.65, враги раз в 300 кадров (5 секунд), максимум 3, скорость 1.0, добавляются движущиеся платформы (20%).

Платформы имеют четыре типа с различной механикой. Зелёные (green) – обычные статичные платформы, с вероятностью 40% получают декоративный элемент (grass1/grass2). Коричневые (brown) – разрушаемые: после приземления игрока платформа трескается с четырьмя стадиями разрушения (crackFrame от 0 до 3) и полностью разрушается через 30 кадров. Синие (blue) – пружинящие: при прыжке с такой платформы игрок получает усиленный импульс (`this.springStrength = -18` вместо обычных `-12`), с вероятностью 30% получают декоративный гриб (mushroom_red). Фиолетовые (purple) – движущиеся: горизонтально перемещаются со скоростью 1–1.5, отражаясь от краёв экрана, с вероятностью 30% получают декоративный гриб (mushroom_brown). Ограничения по уровням: Level 1 – только зелёные; Level 2 – зелёные, коричневые, синие; Level 3 – все четыре типа.

Физика персонажа реализована с параметрами: гравитация 0.5 пикселя/кадр², сила прыжка –12, сила прыжка с пружины –18, горизонтальная скорость 5 пикселей/кадр, затухание горизонтальной скорости $\times 0.8$ при отпуске клавиши. Персонаж имеет хитбокс 30×30 пикселей. При коллизии с платформой сверху (AABB-проверка с условием $v_y \geq 0$) персонаж устанавливается на платформу и получает импульс прыжка. При касании врага теряется 1 жизнь, включается режим неуязвимости на 60 кадров (1 секунда), персонаж отбрасывается вниз ($v_y = 2$), экран трясётся 8 кадров, красная вспышка на 6 кадров, воспроизводится звук hit. Во время неуязвимости персонаж мигает с частотой 4 кадра ($globalAlpha = 0.4$). При обнулении жизней игра завершается со звуком death.

Враги (класс Enemy) имеют размер 28×28 пикселей, перемещаются горизонтально с заданной скоростью, отражаясь от краёв экрана. Спавнятся на платформах выше видимой области экрана ($y = cameraY - 350 \dots -50$). Анимация из двух кадров, смена каждые 15 кадров. Проверка на коллизию с игроком перед спавном исключает появление врага непосредственно на игроке. Монеты (класс Coin) имеют три типа: бронзовые (+10 очков, 35% вероятность), серебряные (+25 очков, 20% вероятность), золотые (+50 очков, 10% вероятность). Анимация из четырёх кадров, смена каждые 6 кадров. Монеты парят с эффектом синусоидального колебания ($bobTimer \times 0.05$, амплитуда 3 пикселя). Размер отображения – 32×32 пикселя с автоматическим масштабированием исходного спрайта.

Бонусы спавнятся на новых платформах и включают джетпак (25% вероятность, 120 кадров активного полёта с $v_y = -3$) и щит (10% вероятность, 1 блокировка врага +50 очков). Джетпак визуально отображается оранжевым кругом на персонаже с тремя языками пламени и эффектом выхлопа снизу (жёлто-оранжевое пламя с случайным мерцанием). Щит отображается голубой пунктирной окружностью вокруг персонажа. Таймер жизни бонусов отображается внизу по центру экрана.

Пользовательский интерфейс включает полноценное главное меню с заголовком «DOODLE JUMP», подзаголовком с предложением выбрать уровень, тремя кнопками уровней (цветное кодирование: зелёный/оранжевый/красный), кнопкой «ГРОМКОСТЬ X%» для доступа к настройкам, отображением текущего рекорда (BEST), навигацией стрелками и мышью. Экран Game Over отображает полупрозрачный тёмный фон, текст «GAME OVER», текущий счёт, лучший результат с отметкой «NEW BEST!», собранные монеты в ряд (иконка x количество), кнопки «RETRY» и «MENU» с навигацией стрелками влево/вправо. Экран настроек содержит ползунок громкости с отображением процентов, кнопки «ПРИМЕНИТЬ» и «ЗАКРЫТЬ», сохранение в localStorage. Экран паузы отображает «PAUSED» (bold 52px) с приостановкой всей игровой логики. HUD (верхняя панель) показывает счёт (Score) и уровень (Level) слева с тёмным полупрозрачным фоном, три иконки сердец (heart_full – красные, heart_empty – серые) справа.

Рендеринг игры построен с вертикальным масштабированием. Фон неба рисуется с помощью линейного градиента из двух цветов (#87CEEB → #E0F7FA), поверх которого отрисовываются 5–6 облаков, псевдослучайно дрейфующих вниз. При тряске экрана (screen shake, вызывается при получении урона) камера смещается на случайную величину в пределах 8 кадров. Текстуры платформ, персонажа, врагов, монет, джетпака, щита и декоративных элементов загружаются из папки assets через класс TextureLoader. Если текстура не загрузилась, используется fallback-отрисовка: цветные прямоугольники для платформ, круги с глазами для врагов, круги с буквами (G/S/B) для монет, синий прямоугольник с глазами для персонажа.

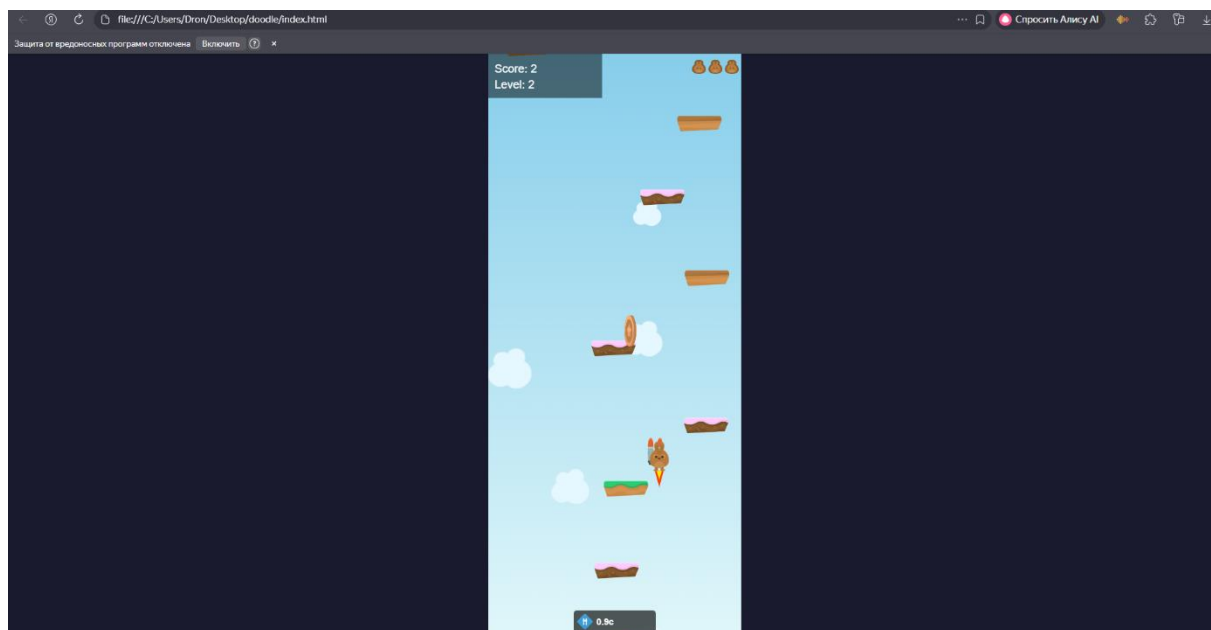


Рисунок 3 – Геймплей разработанной игры

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы были изучены возможности harness-инструментов для разработки программного обеспечения и на практике создана полноценная веб-игра «Doodle Jump» в жанре 2D-вертикального платформера. Игра включает три уровня сложности с возрастающей сложностью, каждый из которых имеет собственную конфигурацию параметров (скорость прокрутки, типы платформ, количество и скорость врагов), систему сбора монет трёх видов (бронзовые, серебряные, золотые) с анимацией парения и пульсации, механику бонусов (джетпак и щит) с таймерами и визуальной индикацией, врагов с процедурным спавном и горизонтальным патрулированием, три жизни с режимом неуязвимости и визуальной обратной связью, а также полноценный пользовательский интерфейс с меню выбора уровня, HUD, экранами Game Over, настроек и паузы.

Технические характеристики разработанного проекта:

- общий объём JavaScript-кода – 5 модулей, около 64 килобайт;
- виртуальный размер игрового мира – 400×600 пикселей;
- количество уровней сложности – 3 (Level 1: простой без движущихся платформ, Level 2: средний с разрушаемыми и пружинящими платформами, Level 3: сложный с движущимися платформами);
- количество анимаций персонажа – 2 состояния (jump1, jump2);
- количество анимаций врага – 2 состояния (fly1, fly2);
- количество анимаций монет – 4 кадра на каждый из трёх типов;
- количество звуковых эффектов – 3 (прыжок, урон, смерть);
- общее количество ассетов (изображения) – 38 PNG-файлов;
- количество типов платформ – 4 (green, brown, blue, purple) с декоративными элементами.

В ходе разработки были реализованы ключевые механики: генерация платформ с возрастанием сложности, физика прыжка с гравитацией и пружинами, коллизии (AABB) с проверкой на приземление сверху, система жизней с неуязвимостью после получения урона, процедурный спавн врагов с проверкой на перекрытие с игроком, вертикальное масштабирование canvas с сохранением чёткости UI, анимированные монеты с плавающей высотой, бонусы с таймерами и визуальной индикацией.

Подход с использованием harness-инструмента (Opencode) показал себя как эффективное решение для разработки многофайловых проектов. Возможность напрямую работать с файловой системой, выполнять команды в терминале, управлять версиями через Git и тестировать изменения в реальном времени значительно ускорила цикл разработки по сравнению с использованием обычных ИИ-веб-чатов. Инструмент самостоятельно отслеживал зависимости между модулями и корректно обновлял все затронутые файлы при внесении изменений. Разработка в режиме Plan mode с последующей реализацией в Build mode позволила структурировать работу и избежать хаотичных изменений.

Опыт показал, что harness-инструменты являются перспективным направлением в автоматизации разработки программного обеспечения. Их использование позволяет разработчику сосредоточиться на архитектурных решениях и креативных аспектах, переложив рутинные операции кодирования, отладки и проверки на ИИ-ассистента. В сочетании с системой контроля версий Git и платформой GitHub для совместной работы и публикации, harness-инструменты образуют мощную экосистему для современной разработки. Данная работа демонстрирует, что разработчики, вооружённые современным harness-инструментом, способны в сжатые сроки создать полноценный игровой продукт, который включает все ключевые элементы: графику, анимацию, физику, звук, управление, интерфейс и систему уровней.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Cursor: [сайт]. – URL: <https://cursor.com/> (дата обращения: 01.06.2026).
2. GitHub: [сайт]. – URL: <https://github.com/> (дата обращения: 01.06.2026).
3. Kenney: [сайт]. – URL: <https://kenney.nl/> (дата обращения: 01.06.2026).
4. Opencode: [сайт]. – URL: <https://opencode.ai/> (дата обращения: 01.06.2026).
5. Sketchfab: [сайт]. – URL: <https://sketchfab.com/> (дата обращения: 01.06.2026).
6. Репозиторий проекта Doodle Jump. – URL: https://github.com/xndxrground/doodle_jump_replica
7. GitHub Pages проекта Doodle Jump. – URL: https://xndxrground.github.io/doodle_jump_replica/